

EDS Practical 4

Name:- Avadhoot R. Huddar

Batch:-M.E.21

PRN:- 202501090087

4.1.1. Pandas - series creation and manipulation

01:30

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the groupby and mean() operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

The user should enter a list of numbers separated by space when prompted.

Output Format:

The program should display the mean of even and odd numbers separately.

Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.

Input:-

```
import pandas as pd
```

```
# Take inputs from the user to create a list of numbers
```

```
numbers = list(map(int, input().split()))
```

```
# Create a Pandas series from the list of numbers
```

```
df=pd.Series(numbers)
```

```
# Grouping by even and odd numbers and calculating the mean
```

```
grouped = df.groupby(df%2==0).mean()
```

```
# Display the mean of even and odd numbers with labels
```

```
grouped.index = ['Even' if is_even else 'Odd' for is_even in grouped.index]
```

```
print("Mean of even and odd numbers:")
```

```
print(grouped)
```

The screenshot shows the CodeTANTRA IDE interface. On the left, there's a sidebar with the title '4.1.1. Pandas - series creation and manipulation'. The main area displays a Python program that takes a list of numbers from the user, creates a Pandas series, and calculates the mean of even and odd numbers separately using the `groupby` and `mean` operations. The program includes input and output formats, and a hint about grouping the series based on whether each number is even or odd. Below the code, there's a 'Sample Test Cases' section with a '+' button. On the right, there's a 'Debugger' panel showing the execution results for three test cases. Each test case shows the expected output, the actual output, and the mean of even and odd numbers. The results indicate that all three test cases passed.

CodeTANTRA Home Learn Anywhere 202501090087@mitaoe.ac.in Support Logout

4.1.1. Pandas - series creation and manipulation 100% 11.00 ms 0.011 s 0.031 s 31.00 ms

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the `groupby` and `mean` () operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.

Sample Test Cases +

Debugger

3 out of 3 shown test case(s) passed
3 out of 3 hidden test case(s) passed

Test case 1 43 ms Debug

Expected output: 1 2 3 4 5 6 7 8 9 10
Actual output: 1 2 3 4 5 6 7 8 9 10
Mean of even and odd numbers: 5.8
Mean of even and odd numbers: 5.8
Odd: 5.8
Odd: 5.8
Even: 6.8
Even: 6.8
dtype: float64
dtype: float64

Test case 2 43 ms Debug

Expected output: 6 8 9 10 2 2 2 10 12 14 16
Actual output: 6 8 9 10 2 2 2 10 12 14 16
Mean of even and odd numbers: 6.83333
Mean of even and odd numbers: 6.83333
Even: 6.83333
Even: 6.83333
dtype: float64
dtype: float64

Test case 3 43 ms Debug

Expected output: 1 5 7 9 11 13 15 17 19 21 25
Actual output: 1 5 7 9 11 13 15 17 19 21 25
Mean of even and odd numbers: 11.5
Mean of even and odd numbers: 11.5
Even: 11.5
Even: 11.5
dtype: float64
dtype: float64

< Prev Reset Submit Next >

4.1.2. Dictionary to dataframe

02:03

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

Convert the dictionary to a Pandas DataFrame.

Add a new row:

Take inputs from the user for the new row data (name, age).

Add the new row to the DataFrame.

Display the DataFrame after adding the new row.

Modify a row:

Modify a specific row by changing the age. Take the row index and new age value from the user.

Display the DataFrame after modifying the row.

Delete a row:

Take the row index to be deleted from the user.

Remove the specified row.

Display the DataFrame after deleting the row.

Add a new column:

Add a column **Gender** with values taken from the user.

Display the DataFrame after adding the new column.

Modify a column:

Convert names to uppercase.

Display the DataFrame after modifying the column.

Delete a column:

Remove the **Age** column.

Display the DataFrame after deleting the column.

Input:-

```
import pandas as pd
```

```
# Provided dictionary of lists
```

```
data = {  
    'Name': ['Alice', 'Bob', 'Charlie'],  
    'Age': [25, 30, 35],  
}
```

```
# Convert the dictionary to a DataFrame
```

```
df = pd.DataFrame(data)
```

```
# Display the original DataFrame
```

```
print("Original DataFrame:")
```

```
print(df)
```

```
# Adding a new row
name = input("New name: ")
age = int(input("New age: "))
new_row = {"Name": name, "Age": age}
df = df.append(new_row, ignore_index = True)
# Display the DataFrame after adding a new row
print("After adding a row:\n",df)
```

```
# Modifying a row
ind = int(input("Index of row to modify: "))
age = int(input("New age: "))

df.loc[ind, "Age"] = age
```

```
# Display the DataFrame after modifying a row
print("After modifying a row:")
print(df)
```

```
# Deleting a row
ind = int(input("Index of row to delete: "))

df = df.drop(index = ind).reset_index(drop = True)
```

```
# Display the DataFrame after deleting a row
```

```
# Deleting a row
```

```
# Display the DataFrame after deleting a row
```

```
print("After deleting a row:")
```

```
print(df)
```

```
# Adding a new column
```

```
gender = input("Enter genders separated by space: ").split()
```

```
df["Gender"] = gender
```

```
# Display the DataFrame after adding a new column
```

```
# Adding a new column
```

```
# Display the DataFrame after adding a new column
```

```
print("After adding a new column:")
```

```
print(df)
```

```
# Modifying a column
```

```
column = []
```

```
for i in df["Name"]:
```

```
    column.append(i.upper())
```

```
df["Name"] = column
```

```
# Display the DataFrame after modifying a column
```

```
# Modifying a column
```

```
# Display the DataFrame after modifying a column
```

```
print("After modifying a column:")
```

```
print(df)
```

```
# Deleting a column
```

```
df = df.drop(columns = ["Age"])
```

```
# Display the DataFrame after deleting a column
```

```
# Deleting a column
```

```
# Display the DataFrame after deleting a column
```

```
print("After deleting a column:")
```

```
print(df)
```

Home
Learn Analytics

2025-10-09 00:07 @mitace.ac.in
Support
Logout
Dashboard

4.1.2. Dictionary to dataframe

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column.

Expected output
Actual output

Original DataFrame:
.....Name: Age
0.....Alice.....25
1.....Bob.....30
2.....Charlie.....35
New name: Susan
New age: 29

After adding a row:
.....Name: Age
0.....Alice.....25
1.....Bob.....30
2.....Charlie.....35
3.....Susan.....29

Index of row to modify:
New age: 27

After modifying a row:
.....Name: Age
0.....Alice.....25
1.....Bob.....30
2.....Charlie.....35
3.....Susan.....29

1 out of 1 shown test case(s) passed
1 out of 1 hidden test case(s) passed

Test case 1
Test
Debug


```
file = input()

data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age",
"Grade"])
```

```
# write your code here..
```

```
print("First five rows:")
```

```
print(data.head())
```

```
avg_age = round(data["Age"].mean(),2)
```

```
print(f"Average age: {avg_age}")
```

```
print("Students with a grade up to B")
```

```
filtered_student = data[data["Grade"] <= "B"]
```

```
print(filtered_student)
```

CDE TANTRA Home Learn Anywhere 202501090087@mitaoe.ac.in Support Logout

4.1.3. Student Information Run Copy Share Fullscreen

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Note:
Refer to the displayed test cases for better understanding.

Sample Test Cases +

Test Results:

Metric	Value
Average time	0.044 s (44.50 ms)
Maximum time	0.063 s (63.00 ms)
Test case 1	1 out of 1 shown test case(s) passed
Hidden test cases	1 out of 1 hidden test case(s) passed

Test case 1 Debug

Expected output	Actual output
studentdata.txt	studentdata.txt
First-Five rows:	First-Five rows:
0: John: 25: A	0: John: 25: A
1: Alin: 22: B	1: Alin: 22: B
2: Emma: 24: A	2: Emma: 24: A
3: Mike: 23: C	3: Mike: 23: C
4: Sony: 21: B	4: Sony: 21: B
Average age: 23.29	Average age: 23.29
Students with a grade up to B:	Students with a grade up to B:
0: John: 25: A	0: John: 25: A
1: Alin: 22: B	1: Alin: 22: B
2: Emma: 24: A	2: Emma: 24: A
4: Sony: 21: B	4: Sony: 21: B
5: Aela: 26: A	5: Aela: 26: A

< Prev Reset Submit Next >

4.2.1. Month with the Highest Total Sales

00:23

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

The CSV file contains the columns: Date, Product, Quantity, Price, and City.

Group the data by Month and calculate the total sales for each month.

Find the month with the highest total sales and display it.

Also, display the total sales for the best month.

Sample Data:

Date,Product,Quantity,Price,City

2025-01-01,Product A,5,20,New York

2025-01-01,Product B,3,15,Los Angeles

2025-01-02,Product A,7,20,New York

2025-01-02,Product C,4,30,Chicago

2025-01-03,Product B,2,15,Chicago

2025-01-03,Product A,8,20,Los Angeles

2025-01-04,Product C,6,30,New York

2025-01-04,Product B,5,15,Los Angeles

2025-01-05,Product A,3,20,Chicago

2025-01-05,Product C,10,30,Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Input:-

```
import pandas as pd
```

```
# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)

df['Date'] = pd.to_datetime(df["Date"])
df["Month"] = df["Date"].dt.strftime("%Y-%m")
df["Total Sales"] = df["Quantity"] * df["Price"]

# Find the month with the highest total sales

sales_by_month = df.groupby("Month")["Total Sales"].sum()
best_month = sales_by_month.idxmax()
highest_sales = sales_by_month.max()

print(f"Best month: {best_month}")
print(f"Total sales: ${highest_sales:.2f}")
```

4.2.1. Month with the Highest Total Sales

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	5	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

monthFor...

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 df['Date'] = pd.to_datetime(df['Date'])
10 df['Month'] = df['Date'].dt.strftime("%Y-%m")
```

Average time

0.028 s

28.00 ms

Maximum time

0.055 s

55.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

Expected output

Actual output

sales_data.csv

sales_data.csv

Best month: 2025-01

Best month: 2025-01

Total sales: \$1230.00

Total sales: \$1230.00

Terminal

Test cases

< Prev

Reset

Submit

Next >

4.2.2. Best Selling Product

00:23

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

The CSV file contains the columns: Date, Product, Quantity, Price, and City.

Find the product that sold the most in terms of quantity sold.

Display the product that sold the most and the total quantity sold for that product.

Sample Data:

Date,Product,Quantity,Price,City

2025-01-01,Product A,5,20,New York

2025-01-01,Product B,3,15,Los Angeles

2025-01-02,Product A,7,20,New York

2025-01-02,Product C,4,30,Chicago

2025-01-03,Product B,2,15,Chicago

2025-01-03,Product A,8,20,Los Angeles

2025-01-04,Product C,6,30,New York

2025-01-04,Product B,5,15,Los Angeles

2025-01-05,Product A,3,20,Chicago

2025-01-05,Product C,10,30,Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Input:-

```
import pandas as pd
```

```
# Prompt the user for the file name
```

```
file_name = input()
```

```
# Load the data
```

```
df = pd.read_csv(file_name)
```

```
product_sales = df.groupby("Product")["Quantity"].sum()
```

```
# Find the product with the highest total quantity sold
```

```
best_product = product_sales.idxmax()
```

```
highest_quantity = product_sales.max()
```

```
# Display the result
```

```
# Display the result
```

```
print(f"Best selling product: {best_product}")

print(f"Total quantity sold: {highest_quantity}")
```

4.2.2. Best Selling Product

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Sample Data:

Date,Product,Quantity,Price,City

2025-01-01,Product A,5,20,New York

2025-01-01,Product B,3,15,Los Angeles

2025-01-02,Product A,7,20,New York

2025-01-02,Product C,4,30,Chicago

2025-01-03,Product B,7,15,Chicago

2025-01-03,Product A,8,20,Los Angeles

2025-01-04,Product C,6,30,New York

2025-01-04,Product B,5,15,Los Angeles

2025-01-05,Product A,3,20,Chicago

2025-01-05,Product C,10,30,Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Average time
0.020 s
20.00 ms

Maximum time
0.042 s
42.00 ms

1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1
Expected output
sales_data.csv
Best selling product: Product A
Total quantity sold: 23

Actual output
sales_data.csv
Best selling product: Product A
Total quantity sold: 23

< Prev

Reset

Submit

Next >

4.2.3. City that Sold the Most Products

00:19

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

The CSV file contains the columns: Date, Product, Quantity, Price, and City.

Group the data by City and calculate the total quantity of products sold for each city.

Find the city that sold the most products (based on the total quantity sold).

Sample Data:

Date,Product,Quantity,Price,City

2025-01-01,Product A,5,20,New York

2025-01-01,Product B,3,15,Los Angeles

2025-01-02,Product A,7,20,New York

2025-01-02,Product C,4,30,Chicago

2025-01-03,Product B,2,15,Chicago

2025-01-03,Product A,8,20,Los Angeles

2025-01-04,Product C,6,30,New York

2025-01-04,Product B,5,15,Los Angeles

2025-01-05,Product A,3,20,Chicago

2025-01-05,Product C,10,30,Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Input:-

```
import pandas as pd
```

```
# Prompt the user for the file name
```

```
file_name = input()
```

```
# Load the data
```

```
df = pd.read_csv(file_name)
```

```
# write the code..
```

```
city_sales = df.groupby("City")["Quantity"].sum()
```

```
best_city=city_sales.idxmax()
```

```
# Display the result
```

```
print(f"City sold the most products: {best_city}")
```

CODETANTRA Home Learn Anywhere 202501090087@mitaoe.ac.in Support Logout

4.2.3. City that Sold the Most Products

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,6,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Average time: 0.020 s (20.33 ms) Maximum time: 0.040 s (40.69 ms)

1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 **Pass** Debug

Expected output: sales_data.csv
City sold the most products: Los Angeles

Actual output: sales_data.csv
City sold the most products: Los Angeles

Terminal Test cases

< Prev Reset Submit Next >

4.2.4. Most Frequently Sold Product Pairs

00:30

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

The CSV file contains the following columns: Date, Product, Quantity, Price, and City.

For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).

Output the product pair/s that was sold most frequently.

Sample Data:

Date,Product,Quantity,Price,City

2025-01-01,Product A,5,20,New York

2025-01-01,Product B,3,15,Los Angeles

2025-01-02,Product A,7,20,New York

2025-01-02,Product C,4,30,Chicago

2025-01-03,Product B,2,15,Chicago

2025-01-03,Product A,8,20,Los Angeles

2025-01-04,Product C,6,30,New York

2025-01-04,Product B,5,15,Los Angeles

2025-01-05,Product A,3,20,Chicago

2025-01-05,Product C,10,30,Los Angeles

Explanation:

Transactions:

2025-01-01: Product A, Product B

2025-01-02: Product A, Product C

2025-01-03: Product B, Product A

2025-01-04: Product C, Product B

2025-01-05: Product A, Product C

Now, let's count how often the pairs of products appear together:

Product A and Product B: Appear in transactions on 2025-01-01 and 2025-01-03.

Product A and Product C: Appear in transactions on 2025-01-02 and 2025-01-05.

Product B and Product C: Appears in transactions on 2025-01-04.

Most Frequent Product Combinations:

Product A and Product B (2 times)

Product A and Product C (2 times)

|

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Input:-

```
import pandas as pd
from itertools import combinations
from collections import Counter

# Prompt user to input the file name
file_name = input()

# Read data from the specified CSV file
df = pd.read_csv(file_name)

date_products = {}

for date, group in df.groupby('Date'):
    products = group['Product'].unique()
    if len(products) > 1:
        date_products[date] = products

# Count product pairs
pair_counter = Counter()

for products in date_products.values():
    # Sort to avoid duplicate pairs like (A, B) and (B, A)
    pairs = combinations(sorted(products), 2)
    pair_counter.update(pairs)
```

```
# Find the maximum frequency
```

```
if pair_counter:
```

```
    max_count = max(pair_counter.values())
```

```
# Output the most frequent product pairs
```

```
for pair, count in pair_counter.items():
```

```
    if count == max_count:
```

```
        print(f"{pair[0]} and {pair[1]}: {count} times")
```

```
else:
```

```
    print("No product pairs found.")
```

```
# write the code
```

```
# Output the most frequent product pairs
```

CODETANTRA Home Learn Anywhere 202501090087@mitaoe.ac.in Support Logout

4.2.4. Most Frequently Sold Product Pairs

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	30	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	5	30	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Explanation:

Transactions:

- 2025-01-01: Product A, Product B
- 2025-01-02: Product A, Product C
- 2025-01-03: Product B, Product A

Test case 1 **100%** **0.022 s** **0.044 s** **1 out of 1 shown test case(s) passed** **2 out of 2 hidden test case(s) passed**

Expected output: sales_data.csv
Actual output: sales_data.csv

Product A and Product B: 2 times
Product A and Product C: 2 times
Product A and Product C: 2 times

Prev Reset Submit Next

4.2.5. Titanic Dataset Analysis and Data Cleaning

00:46

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

Display the first 5 rows of the dataset.

Display the last 5 rows of the dataset.

Get the shape of the dataset (number of rows and columns).

Get a summary of the dataset (using `.info()`).

Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.

Check for missing values and display the count of missing values for each column.

Fill missing values in the 'Age' column with the median age.

Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).

Drop the 'Cabin' column due to many missing values.

Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex

Sample Data:

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S

6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q

7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S

8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S

9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S

10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.

Input:-

```
import pandas as pd
```

```
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
# 1. Display the first 5 rows of the dataset
```

```
print(data.head())
```

```
# 2. Display the last 5 rows of the dataset
```

```
print(data.tail())
```

```
# 3. Get the shape of the dataset (number of rows and columns)
```

```
print(data.shape)
```

```
# 4. Get a summary of the dataset
```

```
print(data.info())
```

```
# 5. Get basic statistics of the dataset
```

```
print(data.describe())
```

```
# 6. Check for missing values and display the count
```

```
print(data.isnull().sum())
```

```
# 7. Fill missing values in the 'Age' column with the median age
```

```
data['Age'].fillna(data['Age'].median(), inplace=True)
```

```
# 8. Fill missing values in the 'Embarked' column with the most frequent value  
(mode)
```

```
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
```

```
# 9. Drop the 'Cabin' column due to many missing values
```

```
data.drop(columns=['Cabin'], inplace=True)
```

10. Create a new column 'FamilySize' by adding 'SibSp' and 'Parch'

```
data['FamilySize'] = data['SibSp'] + data['Parch']
```

CODETANTRA Home Learn Anywhere 202501090087@mitae.ac.in Support Logout

4.2.5. Titanic Dataset Analysis and Data Cleaning

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using `.info()`).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below.

P	S	Pc	N			Si	P	Ti	Fi	C	E
as	ur		a	S	A	b					m

Test case 1 0.218 s 218.00 ms 0.218 s 218.00 ms 1 out of 1 shown test case(s) passed

Expected output: PassengerId - Survived - Pclass, Fare - Cabin - Embarked

Actual output: PassengerId - Survived - Pclass, Fare - Cabin - Embarked

[5 rows x 12 columns]

[5 rows x 12 columns]

Prev Reset Submit Next

4.2.6. Titanic Dataset Analysis and Data Cleaning - 2

00:28

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.

Convert the 'Sex' column to numeric values (male: 0, female: 1).

One-hot encode the 'Embarked' column, dropping the first category.

Get the mean age of passengers.

Get the median fare of passengers.

Get the number of passengers by class.

Get the number of passengers by gender.

Get the number of passengers by survival status.

Calculate the survival rate of passengers.

Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S

6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q

7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S

8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S

9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S

10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.

Input:-

```
import pandas as pd
```

```
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
data['FamilySize'] = data['SibSp'] + data['Parch']
```

```
# 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
```

```
data['IsAlone'] = np.where(data['FamilySize'] == 0, 1, 0)
```

```
# 2. Convert 'Sex' to numeric (male: 0, female: 1)
```

```
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
```

```
# 3. One-hot encode the 'Embarked' column
```

```
data = pd.get_dummies(data, columns=['Embarked'])
```

```
# 4. Get the mean age of passengers
```

```
mean_age = data['Age'].mean()
```

```
print( mean_age)
```

```
# 5. Get the median fare of passengers
```

```
median_fare = data['Fare'].median()
```

```
print( median_fare)
```

6. Get the number of passengers by class

```
print( data['Pclass'].value_counts())
```

7. Get the number of passengers by gender

```
print( data['Sex'].value_counts())
```

8. Get the number of passengers by survival status

```
print( data['Survived'].value_counts())
```

9. Calculate the overall survival rate

```
survival_rate = data['Survived'].mean()
```

```
print(format(survival_rate))
```

10. Calculate the survival rate by gender

```
survival_by_gender = data.groupby('Sex')['Survived'].mean()
```

```
print( survival_by_gender)
```

CODETANTRA

Home Learn Anywhere

202501090087@mitaoe.ac.in Support Logout

4.2.6. Titanic Dataset Analysis and Data Cleaning - 2

00:26

88.00 ms

0.088 s

88.00 ms

1 out of 1 shown test case(s) passed

Test case 1

Debug

Expected output

Actual output

29.69911764705882

29.69911764705882

14.4542

14.4542

3...491

3...491

1...216

1...216

2...184

2...184

Name: Pclass, dtype: int64

Name: Pclass, dtype: int64

0...577

0...577

1...314

1...314

Name: Sex, dtype: int64

Name: Sex, dtype: int64

0...549

0...549

1...342

1...342

Name: Survived, dtype: int64

Name: Survived, dtype: int64

0.3838383838383838

0.3838383838383838

Sex:

Sex:

0...0.188908

0...0.188908

1...0.742038

1...0.742038

Name: Survived, dtype: float64

Name: Survived, dtype: float64

The Titanic dataset contains columns as shown below.

P	S	Pc	N	S	A	Si	P	Ti	Fa	C	E
as	ur	la	a	e	ge	b	ar	ck	re	ab	m
se	vi	ss	m			S	ch	et		in	b
ri	ve									a	
d	d										r

4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

00:26

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

Calculate the survival rate by class.

Calculate the survival rate by embarkation location (Embarked_S).

Calculate the survival rate by family size (FamilySize).

Calculate the survival rate by being alone (IsAlone).

Get the average fare by passenger class (Pclass).

Get the average age by passenger class (Pclass).

Get the average age by survival status (Survived).

Get the average fare by survival status (Survived).

Get the number of survivors by class (Pclass).

Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S

6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q

7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S

8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S

9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S

10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.

Input:-

```
import pandas as pd
```

```
import numpy as np
```

```
# Load the Titanic dataset

data = pd.read_csv('Titanic-Dataset.csv')

data['FamilySize'] = data['SibSp'] + data['Parch']

data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)

data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

print(data.groupby('Pclass') ['Survived'].mean())

#2. Calculate the survival rate by embarked location (Embarked_S)

print(data.groupby ('Embarked_S') ['Survived'].mean())

#3. Calculate the survival rate by family size

print(data.groupby('FamilySize') ['Survived'].mean())

#4. Calculate the survival rate by being alone

print(data.groupby('IsAlone') ['Survived'].mean())

#5. Get the average fare by class

print(data.groupby('Pclass') ['Fare'].mean())
```

#6. Get the average age by class

```
print(data.groupby('Pclass')['Age'].mean())
```

#7. Get the average age by survival status

```
print(data.groupby('Survived')['Age'].mean())
```

#8. Get the average fare by survival status

```
print(data.groupby('Survived')['Fare'].mean())
```

#9. Get the number of survivors by class (sort by values descending)

```
print(data[data['Survived'] == 1]['Pclass'].value_counts())
```

#10. Get the number of non-survivors by class (sort by values descending)

```
print(data[data['Survived'] == 0]['Pclass'].value_counts())
```

[illegible]

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S

6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q

7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S

8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S

9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S

10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.

Input:-

```
import pandas as pd
```

```
import numpy as np
```



```
# Load the Titanic dataset

data = pd.read_csv('Titanic-Dataset.csv')

data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)


# 1. Get the number of survivors by gender

survivors_by_gender = data[data['Survived'] == 1]['Sex'].value_counts()

print(survivors_by_gender)


# 2. Get the number of non-survivors by gender

non_survivors_by_gender = data[data['Survived'] == 0]['Sex'].value_counts()

print(non_survivors_by_gender)


# 3. Get the number of survivors by embarked location (Embarked_S)

survivors_by_embarked_s = data[data['Survived'] == 1]
['Embarked_S'].value_counts()

print(survivors_by_embarked_s)


# 4. Get the number of non-survivors by embarked location (Embarked_S)

non_survivors_by_embarked_s = data[data['Survived'] == 0]
['Embarked_S'].value_counts()

print(non_survivors_by_embarked_s)


# 5. Percentage of children (Age < 18) who survived

children = data[data['Age'] < 18]
```

```
children_survival_rate = children['Survived'].mean()
print(children_survival_rate)
```

6. Percentage of adults (Age >= 18) who survived

```
adults = data[data['Age'] >= 18]
adults_survival_rate = adults['Survived'].mean()
print(adults_survival_rate)
```

7. Median age of survivors

```
median_age_survivors = data[data['Survived'] == 1]['Age'].median()
print(median_age_survivors)
```

8. Median age of non-survivors

```
median_age_non_survivors = data[data['Survived'] == 0]['Age'].median()
print(median_age_non_survivors)
```

9. Median fare of survivors

```
median_fare_survivors = data[data['Survived'] == 1]['Fare'].median()
print(median_fare_survivors)
```

10. Median fare of non-survivors

```
median_fare_non_survivors = data[data['Survived'] == 0]['Fare'].median()
print(median_fare_non_survivors)
```

